Design & Analysis of Algorithms

Lab 1: Algorithm Performance Analysis Workbench

Name: Anshuman Sharma
Roll No: 2501730313
Email: 2501730313@krmu.edu.in

Q1. 1.5 Runtime and Complexity Comparison Table (5 marks)

```
import math

def generate_runtime_complexity_table(n):
    result = []

    linear_search = n
    binary_search = math.floor(math.log2(n)) + 1 if n >= 1 else 0
    bubble_sort = n * (n - 1) // 2
    insertion_sort = n * (n - 1) // 2

    result.append("Runtime Complexity Comparison")
    result.append("Method ObservedCount ExpectedComplexity Observation")
    result.append(f"Linear Search {linear_search} O(n) Grows linearly")
    result.append(f"Binary Search {binary_search} O(log n) Grows logarithmically")
    result.append(f"Bubble Sort {bubble_sort} O(n^2) Grows quadratically")
    result.append(f"Insertion Sort {insertion_sort} O(n^2) Grows quadratically")

    return result
```

Q2. 1.6 Runtime Comparison Chart Data and Scalability Report (5 marks)

```
import math

def generate_runtime_chart_report(sizes):
    result = []

    # ---- Part 1: Runtime Comparison Chart Data ----
    result.append("Runtime Comparison Chart Data")
    result.append("InputSize LinearSearch BinarySearch BubbleSort InsertionSort")

    for n in sizes:
        linear_search = n
        binary_search = math.floor(math.log2(n)) + 1 if n >= 1 else 0
        bubble_sort = n * (n - 1) // 2
        insertion_sort = n * (n - 1) // 2
        result.append(f"{n} {linear_search} {binary_search} {bubble_sort}
{insertion_sort}")

    # ---- Part 2: Scalability Summary ----
    result.append("Scalability Summary")
    result.append("Algorithm Complexity Scalability")
    result.append("Linear Search O(n) Moderate")
    result.append("Binary Search O(log n) Excellent")
    result.append("Bubble Sort O(n^2) Poor")
    result.append("Insertion Sort O(n^2) Poor")

    # ---- Part 3: Key Observations ----
    result.append("Key Observations")
    result.append("Best Algorithm: Binary Search")
    result.append("Most Expensive Algorithm: Bubble Sort")
    result.append("Conclusion: Logarithmic algorithms scale better for large inputs")

    return result
```